Sardar Vallabhbhai National Institute of Technology

Surat – 395007

Department of Artificial Intelligence

Deep Learning (AI302)

Lab Practical – 6

Problem Statement: Design and Implementation of Generative Models (VAE, GAN, CGAN, DCGAN) for Image Generation and Controlled Synthesis on EMNIST and Fashion-MNIST Datasets.

DATASETS FOR EXPERIMENTATION

Students must work with Any of the following datasets:

#	Dataset	Description	Classes	Image Size	Total Samples
1	EMNIST (Balanced Split)	Extended MNIST with handwritten digits and letters	47 balanced classes (digits + letters)	28 × 28 grayscale	~112,800 train / 18,800 test
2	FashionMNIST	Zalando article images – 10 fashion categories	10 classes (T-shirt, Trouser, Pullover, etc.)	28 × 28 grayscale	60,000 train / 10,000 test

---

Problem Statement 1: Variational Autoencoder (VAE)

Design and train a Variational Autoencoder to learn a compact latent representation of EMNIST and Fashion-MNIST images, enabling high-quality image reconstruction and random generation.

Architecture Requirements:

- Encoder Network:
  - Input: 28 × 28 × 1 grayscale image (flattened or with Conv layers)
  - Two FC/Conv branches: one for μ (mean), one for $\log\sigma^2$ (log-variance)
  - Experiment with latent dimensions: $z \in \{8, 16, 32, 64, 128\}$
- Reparameterization Trick:
  - $z = \mu + \varepsilon \times \exp(0.5 \times \log\sigma^2)$, $\varepsilon \sim N(0, I)$
- Decoder Network:
  - Mirror of encoder; output activation: Sigmoid for pixel values $\in [0,1]$

Loss Function:

$\text{ELBO} = E[\log p(x|z)] - KL(q(z|x) \,||\, p(z))$

- Reconstruction Loss: Binary Cross-Entropy or MSE
- KL Divergence: $-0.5 \times \Sigma(1 + \log\sigma^2 - \mu^2 - \sigma^2)$
- KL Annealing: Linearly warm up the KL weight from 0 to 1 over the first 10 epochs

Task to Conduct:

1. Train VAE on both datasets separately and compare reconstruction quality.
2. Latent Space Traversal: Linearly interpolate between two encoded images and visualise intermediate images.
3. Random Sampling: Sample $z \sim N(0, I)$ and generate new images.
4. Latent Dimension Study: Train with $z \in \{8, 32, 128\}$ and compare FID / reconstruction loss.
5. Conditional Latent Control: Force a class label as an auxiliary feature in the latent; generate class-specific samples.

Evaluation Metrics:

Metric	Purpose	Formula / Tool
Reconstruction Loss	Fidelity of decoded images	BCE or MSE
KL Divergence	Latent space regularity	Closed-form KL
FID (Frechet Inception Distance)	Perceptual quality of generated images	torchmetrics.image.FID or pytorch-fid
SSIM	Structural similarity	skimage.metrics.structural_similarity
t-SNE / UMAP of latent space	Cluster separation by class	sklearn / umap-learn

---

Problem Statement 2: Vanilla GAN (Generative Adversarial Network)

Implement the original Goodfellow et al. (2014) GAN framework with a Generator and Discriminator trained in an adversarial minimax game to generate realistic images.

Architecture Requirements:

- Generator (G):
  - Input: Random noise $z \sim N(0, I)$ of dimension 100
  - Fully-connected layers with LeakyReLU activations
  - Output: 784-dim vector reshaped to 28 × 28, Tanh activation
- Discriminator (D):
  - Input: 784-dim flattened image (real or fake)
  - Fully-connected layers with LeakyReLU and Dropout (0.3)
  - Output: single scalar probability (Sigmoid)

Training Protocol:

Minimax objective: $\min_G \max_D V(D,G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$

- Discriminator update: maximize $\log D(x) + \log(1 - D(G(z)))$
- Generator update: maximize $\log D(G(z))$ [non-saturating variant]
- Train D for 1 step, then G for 1 step per iteration
- Label smoothing: use 0.9 for real labels, 0 for fake
- Monitor discriminator accuracy; halt training if D always wins (add noise to labels)

Task to Conduct:

6. Train on EMNIST and Fashion-MNIST and generate image grids every 5 epochs.
7. Mode Collapse Detection: Measure diversity of generated samples using intra-FID across 500 generated images per class.
8. Training Instability Analysis: Plot Generator and Discriminator loss curves; identify oscillations or collapse.
9. Effect of Noise Dimension: Compare $z \in \{50, 100, 200\}$ on sample quality.
10. Gradient Penalty Study: Add gradient penalty $\lambda = 10$ and compare with standard GAN.

Common GAN Failure Modes to Document:

- Mode Collapse: Generator produces same image regardless of noise
- Non-Convergence: D loss $\rightarrow 0$, G loss explodes
- Checkerboard Artifacts: Visible in generated images (mitigate with nearest-neighbour upsampling)

Problem Statement 3: Conditional GAN (CGAN)

Extend the vanilla GAN framework by conditioning both the Generator and Discriminator on class labels, enabling class-controlled image synthesis.

Architecture Requirements:

- Class Conditioning:
  - Embed class label y into a dense vector via nn.Embedding(num_classes, embedding_dim)
  - Concatenate embedding with: noise z (for Generator input) and image features (for Discriminator)
- Generator (G):
  - Input: concat($z \in R^{100}$, embed(y) $\in R^{50}$)
  - FC layers with BatchNorm and ReLU; output 28×28 image (Tanh)
- Discriminator (D):
  - Input: concat(flatten(image) $\in R^{784}$, embed(y) $\in R^{50}$)
  - FC layers with LeakyReLU; output single probability

Task to Conduct:

11. Class-Specific Generation: For each class (0–9 FashionMNIST / 47 EMNIST classes), generate a 5×5 grid of samples and visualise.
12. Label Interpolation: Fix z, vary y from class 0 to class 9, observe smooth transition in generated styles.
13. Mismatched Label Attack: Pass wrong class label to D during testing; measure discrimination accuracy.
14. Cross-Dataset Transfer: Train CGAN on FashionMNIST; fine-tune encoder on EMNIST; compare adaptation speed.
15. Intra-Class Diversity: For a fixed class, generate 100 samples and measure variance in pixel space.

Evaluation:

Metric	EMNIST	FashionMNIST
FID (per class)	Report for each of 47 classes	Report for each of 10 classes
Intra-FID	Diversity within each generated class	Diversity within each generated class
Classifier Accuracy	Train ResNet-18 on generated data, test on real	Same protocol
IS (Inception Score)	Overall image quality + diversity	Overall image quality + diversity

---

Problem Statement 4: Deep Convolutional GAN (DCGAN)

Implement Radford et al. (2015) DCGAN — replacing fully-connected layers with strided convolutions for higher fidelity image generation while stabilising GAN training.

Architecture Requirements (Radford et al. Guidelines):

- Generator (G):
  - Input: z ∈ R^100 → reshape to 1 × 1 × 100 feature map
  - 4 × Transposed Conv (ConvTranspose2d) blocks: 512→256→128→64→1
  - Each block: ConvTranspose2d + BatchNorm + ReLU (final layer: Tanh)
  - For 28 × 28 output: project z to 7×7×128 → two fractional-stride upsamples
- Discriminator (D):
  - 4 × Strided Conv (Conv2d, stride=2) blocks: 1→64→128→256→512
  - Each block: Conv2d + BatchNorm (skip for first layer) + LeakyReLU(0.2)
  - Flatten → FC(1) → Sigmoid
- No Pooling layers (use strided convolutions instead)
- No FC layers in Generator except the initial projection

Task to Conduct:

16. Baseline DCGAN on both EMNIST and FashionMNIST; save generated grids every 5 epochs.
17. Architecture Ablation: Remove BatchNorm from G, from D, from both — compare training stability and FID.
18. Spectral Normalisation: Apply to D; compare with standard DCGAN (affects Lipschitz constraint).
19. Progressive Growing: Start at 7×7, grow to 14×14, then 28×28 — observe training stability gains.
20. Latent Space Arithmetic: Generate images for “a” + “B” − “b” (EMNIST characters) and observe semantic combinations.

Recommended Hyperparameters (Radford et al. 2015):

Hyperparameter	Value	Notes
Batch Size	128	Standard for 28×28 images
Learning Rate (G & D)	0.0002	Use separate optimisers
Adam β1	0.5	Lower than default 0.9 for GAN stability

Hyperparameter	Value	Notes
Adam β2	0.999	Default
LeakyReLU slope	0.2	Discriminator only
Weight Initialisation	N(0, 0.02)	Conv & BatchNorm weights
Epochs	50–100	Monitor FID curve

Problem Statement 5: Comparative Study — VAE vs GAN vs CGAN vs DCGAN

Systematically compare all four generative architectures on both datasets across quality, diversity, controllability, and training stability dimensions.

Comparison Dimensions:

Criterion	VAE	GAN	CGAN	DCGAN
Architecture	Encoder–Decoder	G + D (FC)	Conditional G + D	G + D (Conv)
Latent Space	Structured, continuous	Unstructured	Class-conditioned	Unstructured
Loss Function	ELBO (Recon + KL)	Minimax / BCE	Conditional BCE	BCE (Conv features)
Class Control	Partial (conditional VAE)	None	Full (label conditioning)	None (extend to CGAN)
Training Stability	High (no adversarial)	Low (mode collapse)	Medium	Medium–High
Sample Sharpness	Blurry	Sharp (if converged)	Sharp + controllable	Sharpest
Inference Speed	Fast (single pass)	Fast	Fast	Fast

Quantitative Comparison Table (Fill with Your Results):

Model	Dataset	FID ↓	IS ↑	Recon. MSE ↓	Training Time (GPU hrs)	Epochs to Converge
VAE	EMNIST	—	—	—	—	—
VAE	FashionMNIST	—	—	—	—	—
GAN	EMNIST	—	—	N/A	—	—
GAN	FashionMNIST	—	—	N/A	—	—
CGAN	EMNIST	—	—	N/A	—	—
CGAN	FashionMNIST	—	—	N/A	—	—
DCGAN	EMNIST	—	—	N/A	—	—

Model	Dataset	FID ↓	IS ↑	Recon. MSE ↓	Training Time (GPU hrs)	Epochs to Converge
DCGAN	FashionMNIST	—	—	N/A	—	—

Problem Statement 6: Data Augmentation using Generative Models

Use trained generative models as a data augmentation engine: generate synthetic training samples and measure their utility for downstream classification.

Protocol:

21. Train a simple CNN classifier (LeNet-5 or ResNet-18) on 10% of real FashionMNIST training data (baseline).
22. Augment training set with synthetic images from CGAN / DCGAN to reach 50% real + 50% synthetic.
23. Retrain the same classifier on the augmented dataset.
24. Evaluate both classifiers on the real FashionMNIST test set.
25. Repeat for EMNIST, focusing on under-represented character classes.

Analysis Questions:

- Does synthetic data improve accuracy? By how much?
- Which generative model produces the most useful augmentation data?
- Does augmentation help equally for all classes or mainly minority ones?
- What is the optimal real:synthetic ratio?

Problem Statement 7: Loss Function & Training Stability Analysis

Investigate how different GAN objective functions affect training stability and sample quality on both datasets.

GAN Variants to Implement:

Variant	Loss Function	Key Property
Vanilla GAN	BCE minimax	Baseline; prone to mode collapse
Non-Saturating GAN	G: $-E[\log D(G(z))]$	Better gradients for G early in training
WGAN	Wasserstein distance (weight clipping)	Stable; meaningful loss metric
WGAN-GP	Wasserstein + gradient penalty $\lambda=10$	Improved Lipschitz constraint
LSGAN	Least-squares GAN loss	Reduces vanishing gradients

Analysis to Perform:

- Plot D loss and G loss vs epochs for all variants.

- Compute FID every 10 epochs and plot FID curves.
- Measure training wall-clock time per epoch.
- Identify which variant converges fastest / most stably on each dataset.